```
// ==================== mobile/App.tsx ====================
import React, { useEffect, useState } from 'react'
import { StatusBar } from 'expo-status-bar'
import { ActivityIndicator, View, StyleSheet } from 'react-native'
import { SafeAreaProvider } from 'react-native-safe-area-context'
import { GestureHandlerRootView } from 'react-native-gesture-handler'
import { useAuthStore } from './src/stores/authStore'
import AppNavigator from './src/navigation/AppNavigator'
import ToastContainer from './src/components/Toast'
import ConfirmDialog from './src/components/ConfirmDialog'
import { theme } from './src/theme'

export default function App() {
  const init = useAuthStore((s) => s.init)
  const [ready, setReady] = useState(false)

  useEffect(() => {
    init().finally(() => setReady(true))
  }, [init])

  if (!ready) {
    return (
      <View style={styles.splash}>
        <ActivityIndicator size="large" color={theme.primary} />
      </View>
    )
  }

  return (
    <GestureHandlerRootView style={{ flex: 1 }}>
      <SafeAreaProvider>
        <StatusBar style="dark" />
        <AppNavigator />
        <ToastContainer />
        <ConfirmDialog />
      </SafeAreaProvider>
    </GestureHandlerRootView>
  )
}

const styles = StyleSheet.create({
  splash: { flex: 1, alignItems: 'center', justifyContent: 'center', backgroundColor: theme.bg },
})

// ==================== mobile/src/lib/config.ts ====================
import Constants from 'expo-constants'

export const API_BASE_URL =
  (Constants.expoConfig?.extra as any)?.API_BASE_URL ||
  process.env.EXPO_PUBLIC_API_BASE_URL ||
```

```
  'http://localhost:3000/api'

export const TOKEN_KEY = '@clound_note/token'
export const USER_KEY = '@clound_note/user'

// ==================== mobile/src/lib/storage.ts ====================
import AsyncStorage from '@react-native-async-storage/async-storage'
import { TOKEN_KEY, USER_KEY } from './config'

export async function loadStoredAuth(): Promise<{ token: string | null; user: any | null }> {
  const [token, userJson] = await Promise.all([
    AsyncStorage.getItem(TOKEN_KEY),
    AsyncStorage.getItem(USER_KEY),
  ])
  let user: any = null
  if (userJson) {
    try {
      user = JSON.parse(userJson)
    } catch {
      user = null
    }
  }
  return { token, user }
}

export async function saveAuth(token: string, user: any): Promise<void> {
  await Promise.all([
    AsyncStorage.setItem(TOKEN_KEY, token),
    AsyncStorage.setItem(USER_KEY, JSON.stringify(user)),
  ])
}

export async function clearAuth(): Promise<void> {
  await Promise.all([AsyncStorage.removeItem(TOKEN_KEY), AsyncStorage.removeItem(USER_KEY)])
}

// ==================== mobile/src/lib/api.ts ====================
import { API_BASE_URL } from './config'

export interface ApiError extends Error {
  status?: number
  code?: string
}

type RequestOptions = {
  method?: 'GET' | 'POST' | 'PUT' | 'DELETE'
  body?: any
  query?: Record<string, string | string[] | undefined>
  token?: string | null
```

```
}


function buildUrl(path: string, query?: RequestOptions['query']): string {
  if (!query) return `${API_BASE_URL}${path}`
  const params = Object.entries(query)
    .filter(([, v]) => v !== undefined && v !== '')
    .map(([k, v]) => `${encodeURIComponent(k)}=${encodeURIComponent(String(v))}`)
    .join('&')
  return params ? `${API_BASE_URL}${path}?${params}` : `${API_BASE_URL}${path}`
}

export async function request<T = any>(path: string, opts: RequestOptions = {}): Promise<T> {
  const headers: Record<string, string> = { 'Content-Type': 'application/json' }
  if (opts.token) headers.Authorization = `Bearer ${opts.token}`

  const res = await fetch(buildUrl(path, opts.query), {
    method: opts.method || 'GET',
    headers,
    body: opts.body !== undefined ? JSON.stringify(opts.body) : undefined,
  })

  let data: any = null
  const text = await res.text()
  if (text) {
    try {
      data = JSON.parse(text)
    } catch {
      data = { message: text }
    }
  }

  if (!res.ok) {
    const err = new Error(data?.message || `请求失败 (${res.status})`) as ApiError
    err.status = res.status
    err.code = data?.code
    throw err
  }

  return data as T
}

export const api = {
  auth: {
    register: (body: { email: string; username: string; password: string; nickname?: string }) =>
      request<{ token: string; user: User }>('/auth/register', { method: 'POST', body }),
    login: (body: { account: string; password: string }) =>
      request<{ token: string; user: User }>('/auth/login', { method: 'POST', body }),
    me: (token: string) => request<{ user: User }>('/auth/me', { token }),
    updateProfile: (token: string, body: { nickname?: string; avatar?: string }) =>
```

```
      request<{ user: User }>('/auth/profile', { method: 'PUT', token, body }),
    updatePassword: (token: string, body: { oldPassword: string; newPassword: string }) =>

      request<{ message: string }>('/auth/password', { method: 'PUT', token, body }),
  },
  notes: {
    list: (token: string, params: Record<string, any> = {}) =>
      request<{ items: Note[]; total: number }>('/notes', { token, query: params }),
    get: (token: string, id: string) => request<{ note: Note }>(`/notes/${id}`, { token }),
    create: (token: string, body: Partial<Note> & { tagIds?: string[] }) =>
      request<{ note: Note }>('/notes', { method: 'POST', token, body }),
    update: (token: string, id: string, body: Partial<Note> & { tagIds?: string[] }) =>
      request<{ note: Note }>(`/notes/${id}`, { method: 'PUT', token, body }),
    remove: (token: string, id: string) =>
      request<{ message: string }>(`/notes/${id}`, { method: 'DELETE', token }),
    restore: (token: string, id: string) =>
      request<{ note: Note; message: string }>(`/notes/${id}/restore`, { method: 'POST', token }),
    emptyTrash: (token: string) =>
      request<{ message: string; count: number }>('/notes/trash/empty', { method: 'DELETE', token }),
  },
  tags: {
    list: (token: string) => request<{ tags: Tag[] }>('/tags', { token }),
    create: (token: string, body: { name: string; color?: string }) =>
      request<{ tag: Tag }>('/tags', { method: 'POST', token, body }),
    update: (token: string, id: string, body: { name?: string; color?: string }) =>
      request<{ tag: Tag }>(`/tags/${id}`, { method: 'PUT', token, body }),
    remove: (token: string, id: string) =>
      request<{ message: string }>(`/tags/${id}`, { method: 'DELETE', token }),
  },
}

export interface User {
  id: string
  email: string
  username: string
  nickname?: string | null
  avatar?: string | null
  createdAt?: string
}

export interface Tag {
  id: string
  name: string
  color?: string
  _count?: { notes: number }
}

export interface Note {
  id: string
  title: string
```

```
  content: string
  userId: string
  pinned: boolean

  deletedAt?: string | null
  createdAt: string
  updatedAt: string
  tags?: Tag[]
}

// ==================== mobile/src/theme.ts ====================
export const palette = {
  // 主色 - 蓝紫渐变感
  primary: '#6366f1',
  primaryDark: '#4f46e5',
  primaryLight: '#818cf8',
  primarySoft: '#eef2ff',

  // 辅助色
  accent: '#ec4899',
  success: '#10b981',
  warning: '#f59e0b',
  danger: '#ef4444',
  info: '#0ea5e9',

  // 中性
  bg: '#f5f5f9',
  bgSoft: '#fafafe',
  card: '#ffffff',
  border: '#e7e7ef',
  borderStrong: '#d4d4e0',

  // 文字
  text: '#0f172a',
  textMuted: '#64748b',
  textLight: '#94a3b8',
  textInverse: '#ffffff',

  // 标签色板
  tagColors: ['#6366f1', '#ec4899', '#10b981', '#f59e0b', '#0ea5e9', '#8b5cf6', '#ef4444'],
}

export const spacing = {
  xs: 4,
  sm: 8,
  md: 12,
  lg: 16,
  xl: 20,
  xxl: 28,
}
```

```
export const radius = {
  sm: 6,
  md: 10,

  lg: 14,
  xl: 20,
  pill: 999,
}

export const shadow = {
  sm: { shadowColor: '#1e293b', shadowOffset: { width: 0, height: 1 }, shadowOpacity: 0.05, shadowRadius: 2,
elevation: 1 },
  md: { shadowColor: '#1e293b', shadowOffset: { width: 0, height: 2 }, shadowOpacity: 0.08, shadowRadius: 6,
elevation: 3 },
  lg: { shadowColor: '#4f46e5', shadowOffset: { width: 0, height: 8 }, shadowOpacity: 0.18, shadowRadius: 16,
elevation: 8 },
}

export const theme = {
  ...palette,
  spacing,
  radius,
  shadow,
}

export function timeAgo(iso: string): string {
  const then = new Date(iso).getTime()
  if (Number.isNaN(then)) return ''
  const diff = Math.floor((Date.now() - then) / 1000)
  if (diff < 60) return '刚刚'
  if (diff < 3600) return `${Math.floor(diff / 60)} 分钟前`
  if (diff < 86400) return `${Math.floor(diff / 3600)} 小时前`
  if (diff < 2592000) return `${Math.floor(diff / 86400)} 天前`
  return new Date(iso).toLocaleDateString('zh-CN', { month: '2-digit', day: '2-digit' })
}

export function initials(name?: string | null, email?: string | null): string {
  const src = (name || '').trim() || (email || '').trim()
  if (!src) return '?'
  const ch = src.charAt(0).toUpperCase()
  return ch
}

export function colorFromString(s: string): string {
  let hash = 0
  for (let i = 0; i < s.length; i++) hash = (hash * 31 + s.charCodeAt(i)) | 0
  return palette.tagColors[Math.abs(hash) % palette.tagColors.length]
}

export function countChars(s: string): number {
  return Array.from(s).length
}
```

```
export function countWords(s: string): number {
  const t = s.trim()
  if (!t) return 0
  // 中文按字数 + 英文按空格分词

  const cjk = (t.match(/[\u4e00-\u9fff]/g) || []).length
  const en = (t.replace(/[\u4e00-\u9fff]/g, ' ').split(/\s+/).filter(Boolean)).length
  return cjk + en
}

// ==================== mobile/src/stores/authStore.ts ====================
import { create } from 'zustand'
import { api, type User } from '../lib/api'
import { loadStoredAuth, saveAuth, clearAuth } from '../lib/storage'

interface AuthState {
  token: string | null
  user: User | null
  loading: boolean
  error: string | null
  init: () => Promise<void>
  login: (account: string, password: string) => Promise<void>
  register: (body: { email: string; username: string; password: string; nickname?: string }) => Promise<void>
  logout: () => Promise<void>
  refreshUser: () => Promise<void>
  updateProfile: (body: { nickname?: string; avatar?: string }) => Promise<void>
  clearError: () => void
}

export const useAuthStore = create<AuthState>((set, get) => ({
  token: null,
  user: null,
  loading: false,
  error: null,

  init: async () => {
    const { token, user } = await loadStoredAuth()
    set({ token, user })
    if (token) {
      try {
        const { user: fresh } = await api.auth.me(token)
        set({ user: fresh })
      } catch {
        await clearAuth()
        set({ token: null, user: null })
      }
    }
  },
```

```
  login: async (account, password) => {
    set({ loading: true, error: null })
    try {
      const { token, user } = await api.auth.login({ account, password })
      await saveAuth(token, user)
      set({ token, user, loading: false })

    } catch (e: any) {
      set({ loading: false, error: e.message || '登录失败' })
      throw e
    }
  },

  register: async (body) => {
    set({ loading: true, error: null })
    try {
      const { token, user } = await api.auth.register(body)
      await saveAuth(token, user)
      set({ token, user, loading: false })
    } catch (e: any) {
      set({ loading: false, error: e.message || '注册失败' })
      throw e
    }
  },

  logout: async () => {
    await clearAuth()
    set({ token: null, user: null })
  },

  refreshUser: async () => {
    const token = get().token
    if (!token) return
    const { user } = await api.auth.me(token)
    set({ user })
  },

  updateProfile: async (body) => {
    const token = get().token
    if (!token) throw new Error('未登录')
    const { user } = await api.auth.updateProfile(token, body)
    await saveAuth(token, user)
    set({ user })
  },

  clearError: () => set({ error: null }),
}))

// ==================== mobile/src/stores/noteStore.ts ====================
import { create } from 'zustand'
```

```
import { api, type Note, type Tag } from '../lib/api'

interface NotesState {
  items: Note[]
  tags: Tag[]
  loading: boolean
  error: string | null

  listParams: { trashed: boolean; pinnedOnly: boolean; tagId?: string; keyword?: string }
  fetchList: (token: string, params?: Partial<NotesState['listParams']>) => Promise<void>
  fetchTags: (token: string) => Promise<void>
  createTag: (token: string, name: string) => Promise<void>
  createNote: (token: string, body: Partial<Note> & { tagIds?: string[] }) => Promise<Note>
  updateNote: (token: string, id: string, body: Partial<Note> & { tagIds?: string[] }) => Promise<Note>
  removeNote: (token: string, id: string) => Promise<string>
  restoreNote: (token: string, id: string) => Promise<Note>
  emptyTrash: (token: string) => Promise<number>
  clearError: () => void
}

export const useNotesStore = create<NotesState>((set, get) => ({
  items: [],
  tags: [],
  loading: false,
  error: null,
  listParams: { trashed: false, pinnedOnly: false },

  fetchList: async (token, params) => {
    set({ loading: true, error: null })
    const nextParams = {
      trashed: params?.trashed ?? false,
      pinnedOnly: params?.pinnedOnly ?? false,
      tagId: params?.tagId,
      keyword: params?.keyword,
    }
    set({ listParams: nextParams })
    try {
      const { items, total } = await api.notes.list(token, {
        trashed: String(nextParams.trashed),
        pinnedOnly: String(nextParams.pinnedOnly),
        tagId: nextParams.tagId,
        keyword: nextParams.keyword,
      })
      set({ items, loading: false })
    } catch (e: any) {
      set({ loading: false, error: e.message })
    }
  },

  fetchTags: async (token) => {
```

```
  try {
    const { tags } = await api.tags.list(token)
    set({ tags })
  } catch (e: any) {
    set({ error: e.message })
  }
},


createTag: async (token, name) => {
  try {
    await api.tags.create(token, { name })
    await get().fetchTags(token)
  } catch (e: any) {
    set({ error: e.message })
    throw e
  }
},

createNote: async (token, body) => {
  const { note } = await api.notes.create(token, body)
  set({ items: [note, ...get().items] })
  return note
},

updateNote: async (token, id, body) => {
  const { note } = await api.notes.update(token, id, body)
  set({
    items: get().items.map((n) => (n.id === id ? note : n)),
  })
  return note
},

removeNote: async (token, id) => {
  const { message } = await api.notes.remove(token, id)
  const inTrash = get().listParams.trashed
  set({
    items: inTrash
      ? get().items.filter((n) => n.id !== id)
      : get().items.map((n) =>
          n.id === id ? { ...n, deletedAt: new Date().toISOString() } : n,
        ),
  })
  return message
},

restoreNote: async (token, id) => {
  const { note } = await api.notes.restore(token, id)
  set({ items: get().items.filter((n) => n.id !== id) })
  return note
```

```
  },

  emptyTrash: async (token) => {
    const { count } = await api.notes.emptyTrash(token)
    set({ items: [] })
    return count
  },

  clearError: () => set({ error: null }),

}))

// ==================== mobile/src/stores/toastStore.ts ====================
import { create } from 'zustand'

export type ToastType = 'success' | 'error' | 'info'

export interface ToastItem {
  id: number
  type: ToastType
  message: string
  duration: number
}

interface ToastState {
  toasts: ToastItem[]
  show: (message: string, type?: ToastType, duration?: number) => void
  dismiss: (id: number) => void
}

let nextId = 1

export const useToastStore = create<ToastState>((set, get) => ({
  toasts: [],
  show: (message, type = 'info', duration = 2200) => {
    const id = nextId++
    set((s) => ({ toasts: [...s.toasts, { id, type, message, duration }] }))
    if (duration > 0) {
      setTimeout(() => {
        get().dismiss(id)
      }, duration)
    }
  },
  dismiss: (id) => set((s) => ({ toasts: s.toasts.filter((t) => t.id !== id) })),
}))

export const toast = {
  success: (msg: string, duration?: number) => useToastStore.getState().show(msg, 'success', duration),
  error: (msg: string, duration?: number) => useToastStore.getState().show(msg, 'error', duration ?? 3500),
  info: (msg: string, duration?: number) => useToastStore.getState().show(msg, 'info', duration),
```

```
}

// ==================== mobile/src/stores/confirmStore.ts ====================
import { create } from 'zustand'

export interface ConfirmOptions {
  title?: string
  message?: string
  confirmText?: string
  cancelText?: string

  danger?: boolean
}

interface ConfirmState {
  visible: boolean
  options: ConfirmOptions
  resolver: ((v: boolean) => void) | null
  confirm: (opts: ConfirmOptions) => Promise<boolean>
  resolve: (v: boolean) => void
}

export const useConfirmStore = create<ConfirmState>((set, get) => ({
  visible: false,
  options: {},
  resolver: null,
  confirm: (opts) => {
    return new Promise<boolean>((resolve) => {
      set({ visible: true, options: opts, resolver: resolve })
    })
  },
  resolve: (v) => {
    const r = get().resolver
    set({ visible: false, resolver: null })
    if (r) r(v)
  },
}))

export function confirm(opts: ConfirmOptions): Promise<boolean> {
  return useConfirmStore.getState().confirm(opts)
}

// ==================== mobile/src/components/TagChip.tsx ====================
import React from 'react'
import { View, Text, StyleSheet } from 'react-native'
import { theme } from '../theme'

export default function TagChip({
  name,
  color,
```

```
  selected,
  small,
}: {
  name: string
  color?: string
  selected?: boolean
  small?: boolean
}) {
  const c = color || theme.primary
  return (
    <View

      style={[
        styles.chip,
        { borderColor: c },
        selected && { backgroundColor: c, borderColor: c },
        small && styles.small,
      ]}
    >
      <Text style={[styles.text, { color: selected ? '#fff' : c }, small && styles.textSmall]}>
        {name}
      </Text>
    </View>
  )
}

const styles = StyleSheet.create({
  chip: {
    paddingHorizontal: 12,
    paddingVertical: 5,
    borderRadius: 999,
    borderWidth: 1.2,
    backgroundColor: 'transparent',
  },
  small: { paddingHorizontal: 8, paddingVertical: 2, borderRadius: 6 },
  text: { fontSize: 12, fontWeight: '600' },
  textSmall: { fontSize: 11 },
})

// ==================== mobile/src/components/Toast.tsx ====================
import React from 'react'
import { View, Text, StyleSheet, Pressable, Platform } from 'react-native'
import { useSafeAreaInsets } from 'react-native-safe-area-context'
import { useToastStore, type ToastType } from '../stores/toastStore'
import { theme } from '../theme'

const ICONS: Record<ToastType, string> = { success: '✓', error: '!', info: 'i' }
const COLORS: Record<ToastType, string> = {
  success: theme.success,
  error: theme.danger,
```

```
  info: theme.primary,
}

export default function ToastContainer() {
  const toasts = useToastStore((s) => s.toasts)
  const dismiss = useToastStore((s) => s.dismiss)
  const insets = useSafeAreaInsets()
  return (
    <View
      pointerEvents="box-none"
      style={[styles.wrap, { paddingTop: Math.max(insets.top, 12) + 4 }]}
    >

      {toasts.map((t) => {
        const color = COLORS[t.type]
        return (
          <Pressable
            key={t.id}
            onPress={() => dismiss(t.id)}
            style={({ pressed }) => [
              styles.toast,
              { borderLeftColor: color, opacity: pressed ? 0.85 : 1 },
            ]}
          >
            <View style={[styles.icon, { backgroundColor: color }]}>
              <Text style={styles.iconText}>{ICONS[t.type]}</Text>
            </View>
            <Text style={styles.text}>{t.message}</Text>
          </Pressable>
        )
      })}
    </View>
  )
}

const styles = StyleSheet.create({
  wrap: {
    position: 'absolute',
    top: 0,
    left: 0,
    right: 0,
    alignItems: 'center',
    zIndex: 1000,
    elevation: 1000,
  },
  toast: {
    flexDirection: 'row',
    alignItems: 'center',
    width: '92%',
    maxWidth: 480,
```

```
    backgroundColor: '#fff',
    borderRadius: 12,
    paddingVertical: 12,
    paddingHorizontal: 14,
    marginBottom: 8,
    borderLeftWidth: 4,
    ...Platform.select({
      web: { boxShadow: '0 8px 24px rgba(15,23,42,0.18)' },
      default: { shadowColor: '#1e293b', shadowOpacity: 0.18, shadowRadius: 16, shadowOffset: { width: 0, height:
8 }, elevation: 8 },
    }),
  },
  icon: {
    width: 22,

    height: 22,
    borderRadius: 11,
    alignItems: 'center',
    justifyContent: 'center',
    marginRight: 10,
  },
  iconText: { color: '#fff', fontSize: 14, fontWeight: '700' },
  text: { flex: 1, fontSize: 14, color: theme.text, fontWeight: '500', lineHeight: 20 },
})

// ==================== mobile/src/components/ConfirmDialog.tsx ====================
import React from 'react'
import { View, Text, TouchableOpacity, StyleSheet, Modal, Pressable, ActivityIndicator } from 'react-native'
import { useConfirmStore } from '../stores/confirmStore'
import { theme } from '../theme'

export default function ConfirmDialog() {
  const { visible, options, resolve } = useConfirmStore()
  const [busy, setBusy] = React.useState(false)

  async function handleConfirm() {
    setBusy(true)
    try {
      await resolve(true)
    } finally {
      setBusy(false)
    }
  }

  return (
    <Modal visible={visible} transparent animationType="fade" onRequestClose={() => resolve(false)}>
      <Pressable style={styles.overlay} onPress={() => !busy && resolve(false)}>
        <Pressable style={styles.dialog} onPress={(e) => e.stopPropagation()}>
          {options.title ? <Text style={styles.title}>{options.title}</Text> : null}
          {options.message ? <Text style={styles.message}>{options.message}</Text> : null}
          <View style={styles.actions}>
```

```
            <TouchableOpacity
              style={[styles.btn, styles.cancelBtn]}
              onPress={() => resolve(false)}
              disabled={busy}
            >
              <Text style={styles.cancelText}>{options.cancelText || '取消'}</Text>
            </TouchableOpacity>
            <TouchableOpacity
              style={[
                styles.btn,
                styles.confirmBtn,
                options.danger ? styles.dangerBtn : styles.primaryBtn,
              ]}
              onPress={handleConfirm}

              disabled={busy}
            >
              {busy ? (
                <ActivityIndicator color="#fff" size="small" />
              ) : (
                <Text style={styles.confirmText}>
                  {options.confirmText || '确定'}
                </Text>
              )}
            </TouchableOpacity>
          </View>
        </Pressable>
      </Pressable>
    </Modal>
  )
}

const styles = StyleSheet.create({
  overlay: {
    flex: 1,
    backgroundColor: 'rgba(15, 23, 42, 0.55)',
    justifyContent: 'center',
    alignItems: 'center',
    padding: 24,
    zIndex: 9998,
    elevation: 9998,
  },
  dialog: {
    width: '100%',
    maxWidth: 360,
    backgroundColor: theme.card,
    borderRadius: 16,
    padding: 20,
    zIndex: 9999,
    elevation: 9999,
```

```
  },
  title: {
    fontSize: 17,
    fontWeight: '700',
    color: theme.text,
    textAlign: 'center',
    marginBottom: 8,
  },
  message: {
    fontSize: 14,
    color: theme.textMuted,
    textAlign: 'center',
    lineHeight: 20,
    marginBottom: 18,
  },

  actions: {
    flexDirection: 'row',
    gap: 10,
  },
  btn: {
    flex: 1,
    paddingVertical: 12,
    borderRadius: 10,
    alignItems: 'center',
    justifyContent: 'center',
    minHeight: 44,
  },
  cancelBtn: {
    backgroundColor: theme.bg,
    borderWidth: 1.2,
    borderColor: theme.border,
  },
  confirmBtn: {},
  primaryBtn: { backgroundColor: theme.primary },
  dangerBtn: { backgroundColor: theme.danger },
  cancelText: { color: theme.textMuted, fontWeight: '600', fontSize: 15 },
  confirmText: { color: '#fff', fontWeight: '700', fontSize: 15 },
})

// ==================== mobile/src/components/EmptyState.tsx ====================
import React from 'react'
import { View, Text, StyleSheet } from 'react-native'
import { theme } from '../theme'

export default function EmptyState({
  emoji = '',
  text = '暂无笔记',
  hint,
  action,
```

```
}: {
  emoji?: string
  text?: string
  hint?: string
  action?: React.ReactNode
}) {
  return (
    <View style={styles.wrap}>
      <View style={styles.iconWrap}>
        <Text style={styles.emoji}>{emoji}</Text>
      </View>
      <Text style={styles.text}>{text}</Text>
      {hint ? <Text style={styles.hint}>{hint}</Text> : null}
      {action}
    </View>
  )

}

const styles = StyleSheet.create({
  wrap: { flex: 1, alignItems: 'center', justifyContent: 'center', padding: 40 },
  iconWrap: {
    width: 96,
    height: 96,
    borderRadius: 48,
    backgroundColor: theme.primarySoft,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 16,
  },
  emoji: { fontSize: 44 },
  text: { fontSize: 16, color: theme.text, fontWeight: '600' },
  hint: { marginTop: 6, fontSize: 13, color: theme.textMuted, textAlign: 'center', lineHeight: 20 },
})

// ==================== mobile/src/components/Skeleton.tsx ====================
import React from 'react'
import { View, Text, StyleSheet } from 'react-native'

export function Skeleton({ width, height = 16, radius = 6, style }: { width: number | string; height?: number;
radius?: number; style?: any }) {
  return <View style={[{ width, height, borderRadius: radius, backgroundColor: '#e7e7ef' }, style]} />
}

export function NoteCardSkeleton() {
  return (
    <View style={styles.card}>
      <Skeleton width="70%" height={18} />
      <Skeleton width="100%" height={14} style={{ marginTop: 10 }} />
      <Skeleton width="40%" height={11} style={{ marginTop: 16 }} />
    </View>
```

```
  )
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: '#fff',
    borderRadius: 14,
    padding: 16,
    marginBottom: 10,
    borderWidth: 1,
    borderColor: '#e7e7ef',
  },
})

// ==================== mobile/src/components/Markdown.tsx ====================
import React from 'react'
import { View, Text, StyleSheet } from 'react-native'

import { theme } from '../theme'

type Block =
  | { type: 'h1'; text: string }
  | { type: 'h2'; text: string }
  | { type: 'h3'; text: string }
  | { type: 'quote'; text: string }
  | { type: 'code'; text: string }
  | { type: 'ul'; items: string[] }
  | { type: 'ol'; items: string[] }
  | { type: 'p'; text: string }
  | { type: 'hr' }

function parse(md: string): Block[] {
  const lines = md.replace(/\r\n/g, '\n').split('\n')
  const blocks: Block[] = []
  let i = 0
  let listBuf: string[] = []
  let listType: 'ul' | 'ol' | null = null

  const flush = () => {
    if (listBuf.length && listType) {
      blocks.push({ type: listType, items: listBuf })
      listBuf = []
      listType = null
    }
  }

  while (i < lines.length) {
    const line = lines[i]
    if (!line.trim()) {
      flush()
```

```
      i++
      continue
    }
    if (line.trim() === '---' || line.trim() === '***') {
      flush()
      blocks.push({ type: 'hr' })
      i++
      continue
    }
    const h1 = line.match(/^#\s+(.*)$/)
    if (h1) { flush(); blocks.push({ type: 'h1', text: h1[1] }); i++; continue }
    const h2 = line.match(/^##\s+(.*)$/)
    if (h2) { flush(); blocks.push({ type: 'h2', text: h2[1] }); i++; continue }
    const h3 = line.match(/^###\s+(.*)$/)
    if (h3) { flush(); blocks.push({ type: 'h3', text: h3[1] }); i++; continue }
    const quote = line.match(/^>\s?(.*)$/)
    if (quote) { flush(); blocks.push({ type: 'quote', text: quote[1] }); i++; continue }
    const ul = line.match(/^[-*+]\s+(.*)$/)

    if (ul) { listType = 'ul'; listBuf.push(ul[1]); i++; continue }
    const ol = line.match(/^\d+\.\s+(.*)$/)
    if (ol) { listType = 'ol'; listBuf.push(ol[1]); i++; continue }
    const codeFence = line.match(/^```/)
    if (codeFence) {
      flush()
      const buf: string[] = []
      i++
      while (i < lines.length && !lines[i].match(/^```/)) {
        buf.push(lines[i])
        i++
      }
      i++ // skip closing fence
      blocks.push({ type: 'code', text: buf.join('\n') })
      continue
    }
    flush()
    blocks.push({ type: 'p', text: line })
    i++
  }
  flush()
  return blocks
}

function renderInline(text: string, baseStyle: any, keyPrefix: string) {
  const tokens: React.ReactNode[] = []
  const re = /(\*\*([^*]+)\*\*|\*([^*]+)\*|`([^`]+)`)/g
  let last = 0
  let m: RegExpExecArray | null
  let idx = 0
  while ((m = re.exec(text)) !== null) {
```

```
    if (m.index > last) tokens.push(<Text key={`${keyPrefix}-t${idx++}`} style={baseStyle}>{text.slice(last,
m.index)}</Text>)
    if (m[2]) tokens.push(<Text key={`${keyPrefix}-b${idx++}`} style={[baseStyle, { fontWeight: '700' }]}>{m[2]}
</Text>)
    else if (m[3]) tokens.push(<Text key={`${keyPrefix}-i${idx++}`} style={[baseStyle, { fontStyle: 'italic' }]}>
{m[3]}</Text>)
    else if (m[4]) tokens.push(<Text key={`${keyPrefix}-c${idx++}`} style={[baseStyle, styles.codeInline]}>{m[4]}
</Text>)
    last = m.index + m[0].length
  }
  if (last < text.length) tokens.push(<Text key={`${keyPrefix}-t${idx++}`} style={baseStyle}>{text.slice(last)}
</Text>)
  return <Text>{tokens}</Text>
}

export default function Markdown({ content, style }: { content: string; style?: any }) {
  const blocks = parse(content || '')
  if (!blocks.length) {
    return <Text style={[styles.p, { color: theme.textLight }]}>还没有内容...</Text>
  }
  return (
    <View style={[styles.wrap, style]}>
      {blocks.map((b, i) => {
        const k = `b${i}`

        switch (b.type) {
          case 'h1':
            return <Text key={k} style={styles.h1}>{b.text}</Text>
          case 'h2':
            return <Text key={k} style={styles.h2}>{b.text}</Text>
          case 'h3':
            return <Text key={k} style={styles.h3}>{b.text}</Text>
          case 'quote':
            return (
              <View key={k} style={styles.quoteBox}>
                <Text style={styles.quote}>{renderInline(b.text, styles.quote, k)}</Text>
              </View>
            )
          case 'code':
            return (
              <View key={k} style={styles.codeBox}>
                <Text style={styles.code}>{b.text}</Text>
              </View>
            )
          case 'ul':
            return (
              <View key={k} style={styles.list}>
                {b.items.map((it, j) => (
                  <View key={`${k}-${j}`} style={styles.li}>
                    <Text style={styles.bullet}>•</Text>
                    <Text style={styles.liText}>{renderInline(it, styles.liText, `${k}-${j}`)}</Text>
                  </View>
                ))}
              </View>
            )
```

```
          case 'ol':
            return (
              <View key={k} style={styles.list}>
                {b.items.map((it, j) => (
                  <View key={`${k}-${j}`} style={styles.li}>
                    <Text style={styles.bullet}>{j + 1}.</Text>
                    <Text style={styles.liText}>{renderInline(it, styles.liText, `${k}-${j}`)}</Text>
                  </View>
                ))}
              </View>
            )
          case 'hr':
            return <View key={k} style={styles.hr} />
          default:
            return <Text key={k} style={styles.p}>{renderInline(b.text, styles.p, k)}</Text>
        }
      })}
    </View>
  )
}


const styles = StyleSheet.create({
  wrap: { gap: 10 },
  h1: { fontSize: 24, fontWeight: '700', color: theme.text, marginTop: 8 },
  h2: { fontSize: 20, fontWeight: '700', color: theme.text, marginTop: 6 },
  h3: { fontSize: 17, fontWeight: '600', color: theme.text, marginTop: 4 },
  p: { fontSize: 15, lineHeight: 24, color: theme.text },
  quoteBox: {
    borderLeftWidth: 3,
    borderLeftColor: theme.primary,
    paddingLeft: 12,
    paddingVertical: 4,
    backgroundColor: theme.primarySoft,
    borderRadius: 4,
  },
  quote: { fontSize: 14, color: theme.textMuted, lineHeight: 22, fontStyle: 'italic' },
  codeBox: {
    backgroundColor: '#1e293b',
    padding: 12,
    borderRadius: 8,
    marginVertical: 2,
  },
  code: { fontFamily: 'monospace', fontSize: 13, color: '#e2e8f0', lineHeight: 20 },
  codeInline: { fontFamily: 'monospace', fontSize: 13, backgroundColor: theme.primarySoft, color:
theme.primaryDark, paddingHorizontal: 4, borderRadius: 4 },
  list: { gap: 4, paddingLeft: 4 },
  li: { flexDirection: 'row', gap: 8 },
  bullet: { color: theme.primary, fontWeight: '700', fontSize: 15, lineHeight: 22 },
  liText: { flex: 1, fontSize: 15, lineHeight: 22, color: theme.text },
  hr: { height: 1, backgroundColor: theme.border, marginVertical: 8 },
```

```
})

// ==================== mobile/src/components/NoteCard.tsx ====================
import React from 'react'
import { View, Text, StyleSheet } from 'react-native'
import type { Note } from '../lib/api'
import { theme, timeAgo } from '../theme'

interface Props {
  note: Note
  showActions?: boolean
}

function escapePreview(s: string): string {
  return s
    .replace(/^#{1,6}\s+/gm, '')
    .replace(/^>\s?/gm, '')
    .replace(/^[-*+]\s+/gm, '')
    .replace(/^\d+\.\s+/gm, '')
    .replace(/[*`_~]/g, '')
    .replace(/\n+/g, ' ')

    .trim()
}

export default function NoteCard({ note, showActions = false }: Props) {
  const preview = escapePreview(note.content).slice(0, 80)
  const hasTags = note.tags && note.tags.length > 0
  return (
    <View style={[styles.card, note.pinned && styles.cardPinned]}>
      <View style={styles.header}>
        {note.pinned && <Text style={styles.pin}>★</Text>}
        <Text style={styles.title} numberOfLines={1}>
          {note.title || '无标题'}
        </Text>
      </View>

      {preview ? (
        <Text style={styles.preview} numberOfLines={2}>
          {preview}
        </Text>
      ) : (
        <Text style={styles.previewEmpty}>空笔记</Text>
      )}

      <View style={styles.footer}>
        <View style={styles.tags}>
          {hasTags && note.tags!.slice(0, 3).map((t) => (
            <View
              key={t.id}
```

```
              style={[styles.tagChip, { backgroundColor: (t.color || theme.primary) + '22' }]}
            >
              <Text style={[styles.tagText, { color: t.color || theme.primary }]}>
                {t.name}
              </Text>
            </View>
          ))}
          {hasTags && note.tags!.length > 3 && (
            <Text style={styles.tagMore}>+{note.tags!.length - 3}</Text>
          )}
        </View>
        <Text style={styles.time}>{timeAgo(note.updatedAt)}</Text>
      </View>
    </View>
  )
}

const styles = StyleSheet.create({
  card: {
    backgroundColor: theme.card,
    borderRadius: theme.radius.lg,
    padding: 16,

    marginBottom: 10,
    borderWidth: 1,
    borderColor: theme.border,
    ...theme.shadow.sm,
  },
  cardPinned: {
    borderColor: theme.warning + '55',
    backgroundColor: '#fffbeb',
  },
  header: { flexDirection: 'row', alignItems: 'center', gap: 6 },
  pin: { color: theme.warning, fontSize: 16 },
  title: { flex: 1, fontSize: 16, fontWeight: '600', color: theme.text, letterSpacing: 0.2 },
  preview: { marginTop: 6, color: theme.textMuted, fontSize: 13, lineHeight: 20 },
  previewEmpty: { marginTop: 6, color: theme.textLight, fontSize: 13, fontStyle: 'italic' },
  footer: {
    marginTop: 12,
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  tags: { flexDirection: 'row', flex: 1, gap: 6, flexWrap: 'wrap' },
  tagChip: {
    paddingHorizontal: 8,
    paddingVertical: 3,
    borderRadius: 6,
  },
  tagText: { fontSize: 11, fontWeight: '600' },
```

```
  tagMore: { fontSize: 11, color: theme.textLight, alignSelf: 'center' },
  time: { color: theme.textLight, fontSize: 11 },
})

// ==================== mobile/src/navigation/AppNavigator.tsx ====================
import React from 'react'
import { NavigationContainer } from '@react-navigation/native'
import { createNativeStackNavigator, type NativeStackNavigationOptions } from '@react-navigation/native-stack'
import { createBottomTabNavigator } from '@react-navigation/bottom-tabs'
import { Text, View, StyleSheet } from 'react-native'
import { useAuthStore } from '../stores/authStore'
import LoginScreen from '../screens/LoginScreen'
import RegisterScreen from '../screens/RegisterScreen'
import NotesListScreen from '../screens/NotesListScreen'
import NoteEditorScreen from '../screens/NoteEditorScreen'
import TrashScreen from '../screens/TrashScreen'
import SettingsScreen from '../screens/SettingsScreen'
import { theme } from '../theme'

export type RootStackParamList = {
  Main: undefined
  NoteEditor: { id?: string } | undefined
  Login: undefined

  Register: undefined
}

const Stack = createNativeStackNavigator<RootStackParamList>()
const Tab = createBottomTabNavigator()

const screenOptions: NativeStackNavigationOptions = {
  headerBackTitle: '返回',
  headerTitleStyle: { fontWeight: '700', color: theme.text, fontSize: 17 },
  headerShadowVisible: false,
  headerStyle: { backgroundColor: theme.card },
  headerTintColor: theme.primary,
}

function MainTabs() {
  return (
    <Tab.Navigator
      screenOptions={{
        headerShown: false,
        tabBarActiveTintColor: theme.primary,
        tabBarInactiveTintColor: theme.textLight,
        tabBarStyle: {
          backgroundColor: theme.card,
          borderTopColor: theme.border,
          borderTopWidth: 1,
          height: 60,
```

```
          paddingBottom: 6,
          paddingTop: 4,
        },
        tabBarLabelStyle: { fontSize: 11, fontWeight: '600' },
      }}
    >
      <Tab.Screen
        name="Notes"
        component={NotesListScreen}
        options={{
          headerTitle: '我的笔记',
          headerTitleAlign: 'center',
          headerShadowVisible: false,
          headerStyle: { backgroundColor: theme.card },
          headerTitleStyle: { fontWeight: '700', color: theme.text, fontSize: 17 },
          tabBarIcon: ({ focused }) => (
            <Text style={{ fontSize: 20, color: focused ? theme.primary : theme.textLight }}>{focused ? '' :
''}</Text>
          ),
        }}
      />
      <Tab.Screen
        name="Trash"
        component={TrashScreen}
        options={{

          headerTitle: '回收站',
          headerTitleAlign: 'center',
          headerShadowVisible: false,
          headerStyle: { backgroundColor: theme.card },
          headerTitleStyle: { fontWeight: '700', color: theme.text, fontSize: 17 },
          tabBarIcon: ({ focused }) => (
            <Text style={{ fontSize: 20, color: focused ? theme.primary : theme.textLight }}>{focused ? '' :
''}</Text>
          ),
        }}
      />
      <Tab.Screen
        name="Settings"
        component={SettingsScreen}
        options={{
          headerTitle: '我的',
          headerTitleAlign: 'center',
          headerShadowVisible: false,
          headerStyle: { backgroundColor: theme.card },
          headerTitleStyle: { fontWeight: '700', color: theme.text, fontSize: 17 },
          tabBarIcon: ({ focused }) => (
            <Text style={{ fontSize: 20, color: focused ? theme.primary : theme.textLight }}>{focused ? '' :
'⚙'}</Text>
          ),
        }}
      />
    </Tab.Navigator>
```

```
  )
}

export default function AppNavigator() {
  const token = useAuthStore((s) => s.token)
  return (
    <NavigationContainer>
      <Stack.Navigator screenOptions={screenOptions}>
        {token ? (
          <>
            <Stack.Screen name="Main" component={MainTabs} options={{ headerShown: false }} />
            <Stack.Screen name="NoteEditor" component={NoteEditorScreen} options={{ title: '编辑笔记' }} />
          </>
        ) : (
          <>
            <Stack.Screen name="Login" component={LoginScreen} options={{ headerShown: false }} />
            <Stack.Screen name="Register" component={RegisterScreen} options={{ title: '注册账号' }} />
          </>
        )}
      </Stack.Navigator>
    </NavigationContainer>
  )
}

// ==================== mobile/src/screens/LoginScreen.tsx ====================

import React, { useState } from 'react'
import { View, Text, TextInput, TouchableOpacity, StyleSheet, KeyboardAvoidingView, Platform, ScrollView } from
'react-native'
import type { NativeStackScreenProps } from '@react-navigation/native-stack'
import { useAuthStore } from '../stores/authStore'
import { toast } from '../stores/toastStore'
import type { RootStackParamList } from '../navigation/AppNavigator'
import { theme } from '../theme'

type Props = NativeStackScreenProps<RootStackParamList, 'Login'>

export default function LoginScreen({ navigation }: Props) {
  const login = useAuthStore((s) => s.login)
  const loading = useAuthStore((s) => s.loading)
  const [account, setAccount] = useState('')
  const [password, setPassword] = useState('')
  const [showPwd, setShowPwd] = useState(false)

  async function submit() {
    if (!account || !password) return toast.error('请输入账号和密码')
    try {
      await login(account, password)
      toast.success('登录成功')
    } catch (e: any) {
      toast.error('登录失败：' + e.message)
```

```
    }
  }

  return (
    <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.wrap}>
      <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">
        <View style={styles.hero}>
          <View style={styles.logo}>
            <Text style={styles.logoEmoji}>📒</Text>
          </View>
          <Text style={styles.brand}>富商笔记</Text>
          <Text style={styles.slogan}>记录每一刻灵感 · 随时随地同步</Text>
        </View>

        <View style={styles.form}>
          <Text style={styles.label}>邮箱或用户名</Text>
          <View style={styles.inputWrap}>
            <Text style={styles.inputIcon}>✉</Text>
            <TextInput
              style={styles.input}
              placeholder="you@example.com"
              placeholderTextColor={theme.textLight}
              value={account}
              autoCapitalize="none"
              autoCorrect={false}
              onChangeText={setAccount}

            />
          </View>

          <Text style={styles.label}>密码</Text>
          <View style={styles.inputWrap}>
            <Text style={styles.inputIcon}>🔒</Text>
            <TextInput
              style={styles.input}
              placeholder="••••••"
              placeholderTextColor={theme.textLight}
              value={password}
              secureTextEntry={!showPwd}
              onChangeText={setPassword}
            />
            <TouchableOpacity onPress={() => setShowPwd((v) => !v)} style={styles.eyeBtn}>
              <Text style={styles.eyeIcon}>{showPwd ? '🙈' : '👁'}</Text>
            </TouchableOpacity>
          </View>

          <TouchableOpacity style={[styles.btn, loading && styles.btnDisabled]} onPress={submit} disabled=
{loading} activeOpacity={0.85}>
            <Text style={styles.btnText}>{loading ? '登录中…' : '登 录'}</Text>
          </TouchableOpacity>
```

```
          <TouchableOpacity onPress={() => navigation.navigate('Register')} style={styles.linkBtn}>
            <Text style={styles.linkHint}>还没有账号？<Text style={styles.linkText}>立即注册 →</Text></Text>
          </TouchableOpacity>

          <View style={styles.demoTip}>
            <Text style={styles.demoTipText}>
              演示账号：demo@clound.note / 123456
            </Text>
          </View>
        </View>
      </ScrollView>
    </KeyboardAvoidingView>
  )
}

const styles = StyleSheet.create({
  wrap: { flex: 1, backgroundColor: theme.bg },
  scroll: { flexGrow: 1, padding: 24, paddingTop: 60 },
  hero: { alignItems: 'center', marginBottom: 40 },
  logo: {
    width: 88,
    height: 88,
    borderRadius: 28,
    backgroundColor: theme.primary,
    alignItems: 'center',
    justifyContent: 'center',
    marginBottom: 16,

    ...theme.shadow.lg,
  },
  logoEmoji: { fontSize: 44 },
  brand: { fontSize: 30, fontWeight: '800', color: theme.text, letterSpacing: 1 },
  slogan: { marginTop: 8, color: theme.textMuted, fontSize: 13 },
  form: {},
  label: { fontSize: 13, color: theme.textMuted, marginBottom: 8, marginTop: 16, fontWeight: '500' },
  inputWrap: {
    flexDirection: 'row',
    alignItems: 'center',
    backgroundColor: theme.card,
    borderWidth: 1.5,
    borderColor: theme.border,
    borderRadius: theme.radius.md,
    paddingHorizontal: 14,
    height: 50,
  },
  inputIcon: { fontSize: 16, color: theme.textLight, marginRight: 10 },
  input: { flex: 1, fontSize: 15, color: theme.text, paddingVertical: 0 },
  eyeBtn: { paddingHorizontal: 6 },
  eyeIcon: { fontSize: 16 },
  btn: {
```

```
    backgroundColor: theme.primary,
    borderRadius: theme.radius.md,
    paddingVertical: 14,
    alignItems: 'center',
    marginTop: 28,
    ...theme.shadow.md,
  },
  btnDisabled: { opacity: 0.6 },
  btnText: { color: '#fff', fontSize: 16, fontWeight: '700', letterSpacing: 2 },
  linkBtn: { alignItems: 'center', paddingVertical: 16 },
  linkHint: { color: theme.textMuted, fontSize: 13 },
  linkText: { color: theme.primary, fontWeight: '600' },
  demoTip: {
    marginTop: 8,
    padding: 12,
    backgroundColor: theme.primarySoft,
    borderRadius: theme.radius.md,
    alignItems: 'center',
  },
  demoTipText: { color: theme.primaryDark, fontSize: 12 },
})

// ==================== mobile/src/screens/RegisterScreen.tsx ====================
import React, { useState } from 'react'
import { View, Text, TextInput, TouchableOpacity, StyleSheet, KeyboardAvoidingView, Platform, ScrollView } from
'react-native'
import type { NativeStackScreenProps } from '@react-navigation/native-stack'
import { useAuthStore } from '../stores/authStore'
import { toast } from '../stores/toastStore'

import type { RootStackParamList } from '../navigation/AppNavigator'
import { theme } from '../theme'

type Props = NativeStackScreenProps<RootStackParamList, 'Register'>

export default function RegisterScreen({ navigation }: Props) {
  const register = useAuthStore((s) => s.register)
  const loading = useAuthStore((s) => s.loading)
  const [email, setEmail] = useState('')
  const [username, setUsername] = useState('')
  const [nickname, setNickname] = useState('')
  const [password, setPassword] = useState('')
  const [confirm, setConfirm] = useState('')

  function strength(pwd: string): { score: number; label: string; color: string } {
    if (!pwd) return { score: 0, label: '', color: theme.border }
    if (pwd.length < 6) return { score: 1, label: '太短', color: theme.danger }
    let s = 1
    if (/[A-Z]/.test(pwd)) s++
    if (/\d/.test(pwd)) s++
    if (/[^a-zA-Z0-9]/.test(pwd)) s++
```

```
  if (pwd.length >= 10) s++
  const map = [
    { label: '弱', color: theme.danger },
    { label: '一般', color: theme.warning },
    { label: '良好', color: theme.info },
    { label: '强', color: theme.success },
    { label: '极强', color: theme.success },
  ]
  return { score: s, ...map[s - 1] }
}

const pwdScore = strength(password)

async function submit() {
  if (!email || !username || !password) return toast.error('请填写必填项')
  if (password.length < 6) return toast.error('密码至少 6 位')
  if (password !== confirm) return toast.error('两次密码不一致')
  try {
    await register({ email, username, password, nickname: nickname || undefined })
    toast.success('注册成功，已自动登录')
  } catch (e: any) {
    toast.error('注册失败：' + e.message)
  }
}

return (
  <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.wrap}>
    <ScrollView contentContainerStyle={styles.scroll} keyboardShouldPersistTaps="handled">
      <View style={styles.hero}>

    danger: true,
  })
  if (!ok) return
  try {
    await removeNote(token, id!)
    try { await AsyncStorage.removeItem(draftKey(userId, id)) } catch {}
    toast.success('已移入回收站')
    navigation.goBack()
  } catch (e: any) {
    toast.error('删除失败：' + e.message)
  }
}

async function onDuplicate() {
  setShowMenu(false)
  try {
    await createNote(token, { title: title + ' (副本)', content, pinned: false, tagIds: selectedTagIds })
    toast.success('已复制为新笔记')
  } catch (e: any) {
    toast.error('复制失败：' + e.message)
```

```
  }
}

function onCopyContent() {
  setShowMenu(false)
  const full = `# ${title}\n\n${content}`
  try {
    // @ts-ignore
    if (typeof navigator !== 'undefined' && navigator.clipboard) {
      // @ts-ignore
      navigator.clipboard.writeText(full)
      toast.success('笔记内容已复制到剪贴板')
    } else {
      toast.info('当前平台不支持复制，请手动选择文本')
    }
  } catch (e: any) {
    toast.error('复制失败：' + e.message)
  }
}

function toggleTag(tagId: string) {
  setSelectedTagIds((prev) =>
    prev.includes(tagId) ? prev.filter((t) => t !== tagId) : [...prev, tagId],
  )
}

const wordCount = countWords(content)
const charCount = countChars(content)

return (

  <KeyboardAvoidingView behavior={Platform.OS === 'ios' ? 'padding' : undefined} style={styles.wrap}>
    <View style={styles.toolbar}>
      <TouchableOpacity onPress={() => setPinned((p) => !p)} style={styles.toolBtn}>
        <Text style={[styles.pinEmoji, pinned && styles.pinActive]}>{pinned ? '★' : '☆'}</Text>
        <Text style={styles.toolLabel}>{pinned ? '已收藏' : '收藏'}</Text>
      </TouchableOpacity>
      <View style={styles.modeSwitch}>
        <TouchableOpacity
          style={[styles.modeBtn, mode === 'edit' && styles.modeBtnActive]}
          onPress={() => setMode('edit')}
        >
          <Text style={[styles.modeText, mode === 'edit' && styles.modeTextActive]}>编辑</Text>
        </TouchableOpacity>
        <TouchableOpacity
          style={[styles.modeBtn, mode === 'preview' && styles.modeBtnActive]}
          onPress={() => setMode('preview')}
        >
          <Text style={[styles.modeText, mode === 'preview' && styles.modeTextActive]}>预览</Text>
        </TouchableOpacity>
```

```
        </View>
        <TouchableOpacity onPress={() => setShowTagSheet(true)} style={styles.toolBtn}>
          <Text style={styles.pinEmoji}>🏷</Text>
          <Text style={styles.toolLabel}>{selectedTagIds.length || '标签'}</Text>
        </TouchableOpacity>
      </View>

      {mode === 'preview' ? (
        <ScrollView style={styles.preview} contentContainerStyle={styles.previewContent}>
          {title ? <Text style={styles.previewTitle}>{title}</Text> : null}
          <Markdown content={content} />
        </ScrollView>
      ) : (
        <ScrollView style={styles.editor} contentContainerStyle={styles.editorContent}
keyboardShouldPersistTaps="handled">
          <TextInput
            style={styles.title}
            placeholder="标题"
            placeholderTextColor={theme.textLight}
            value={title}
            onChangeText={setTitle}
            maxLength={200}
          />
          <TextInput
            style={styles.content}
            placeholder="在这里记录你的想法...支持 Markdown 语法"
            placeholderTextColor={theme.textLight}
            value={content}
            onChangeText={setContent}
            multiline
            textAlignVertical="top"
          />

        </ScrollView>
      )}

      <View style={styles.statusBar}>
        <Text style={styles.statusText}>{wordCount} 字 • {charCount} 字符</Text>
        <Text style={styles.statusText}>{selectedTagIds.length} 标签</Text>
      </View>

      {/* 更多菜单 */}
      <Modal visible={showMenu} transparent animationType="fade" onRequestClose={() => setShowMenu(false)}>
        <Pressable style={styles.modalOverlay} onPress={() => setShowMenu(false)}>
          <Pressable style={styles.actionSheet} onPress={(e) => e.stopPropagation()}>
            <TouchableOpacity style={styles.actionItem} onPress={() => { setShowMenu(false); save() }}>
              <Text style={styles.actionEmoji}>💾</Text>
              <Text style={styles.actionText}>保存笔记</Text>
            </TouchableOpacity>
            <TouchableOpacity style={styles.actionItem} onPress={onDuplicate}>
              <Text style={styles.actionEmoji}>📋</Text>
```

```
              <Text style={styles.actionText}>复制为新笔记</Text>
            </TouchableOpacity>
            <TouchableOpacity style={styles.actionItem} onPress={onCopyContent}>
              <Text style={styles.actionEmoji}>✂️</Text>
              <Text style={styles.actionText}>复制正文到剪贴板</Text>
            </TouchableOpacity>
            {id ? (
              <TouchableOpacity style={[styles.actionItem, styles.actionDanger]} onPress={onDelete}>
                <Text style={styles.actionEmoji}>🗑</Text>
                <Text style={[styles.actionText, { color: theme.danger }]}>删除（移入回收站）</Text>
              </TouchableOpacity>
            ) : null}
          </Pressable>
        </Pressable>
      </Modal>

      {/* 标签选择 sheet */}
      <Modal visible={showTagSheet} transparent animationType="slide" onRequestClose={() =>
setShowTagSheet(false)}>
        <Pressable style={styles.modalOverlay} onPress={() => setShowTagSheet(false)}>
          <Pressable style={styles.tagSheet} onPress={(e) => e.stopPropagation()}>
            <View style={styles.sheetHeader}>
              <Text style={styles.sheetTitle}>选择标签</Text>
              <Text style={styles.sheetHint}>点击勾选 / 取消</Text>
            </View>
            <View style={styles.tagsList}>
              {tags.map((t) => {
                const selected = selectedTagIds.includes(t.id)
                return (
                  <TouchableOpacity key={t.id} onPress={() => toggleTag(t.id)}>
                    <TagChip name={t.name} color={t.color} selected={selected} />
                  </TouchableOpacity>
                )

              })}
              {tags.length === 0 && <Text style={styles.emptyTags}>还没有标签，可在「我的 → 标签管理」添加
</Text>}
            </View>
            <View style={styles.addTagRow}>
              <TextInput
                style={styles.addTagInput}
                placeholder="新标签名"
                placeholderTextColor={theme.textLight}
                value={newTag}
                onChangeText={setNewTag}
              />
              <TouchableOpacity
                style={styles.addTagBtn}
                onPress={async () => {
                  const name = newTag.trim()
                  if (!name) return
                  try {
```

```
                    const { tag } = await api.tags.create(token, { name })
                    await fetchTags(token)
                    setSelectedTagIds((p) => [...p, tag.id])
                    setNewTag('')
                    toast.success('已添加标签：' + name)
                  } catch (e: any) {
                    toast.error('添加失败：' + e.message)
                  }
                }}
              >
                <Text style={styles.addTagBtnText}>添加</Text>
              </TouchableOpacity>
            </View>
          </Pressable>
        </Pressable>
      </Modal>
    </KeyboardAvoidingView>
  )
}

const styles = StyleSheet.create({
  wrap: { flex: 1, backgroundColor: theme.card },
  headerBtn: { paddingHorizontal: 8, paddingVertical: 4 },
  saveText: { color: theme.primary, fontSize: 16, fontWeight: '700' },
  menuIcon: { color: theme.text, fontSize: 22, fontWeight: '700', marginTop: -6 },
  toolbar: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    paddingHorizontal: 12,
    paddingVertical: 10,
    borderBottomWidth: 1,
    borderBottomColor: theme.border,

    backgroundColor: theme.bgSoft,
  },
  toolBtn: { alignItems: 'center', gap: 2, padding: 4, minWidth: 56 },
  pinEmoji: { fontSize: 20, color: theme.textLight },
  pinActive: { color: theme.warning },
  toolLabel: { fontSize: 11, color: theme.textMuted },
  modeSwitch: {
    flexDirection: 'row',
    backgroundColor: theme.bg,
    borderRadius: 999,
    padding: 3,
    borderWidth: 1,
    borderColor: theme.border,
  },
  modeBtn: { paddingHorizontal: 16, paddingVertical: 6, borderRadius: 999 },
  modeBtnActive: { backgroundColor: theme.primary },
```

```
  modeText: { fontSize: 13, color: theme.textMuted, fontWeight: '500' },
  modeTextActive: { color: '#fff', fontWeight: '700' },
  editor: { flex: 1, backgroundColor: theme.card },
  editorContent: { padding: 20, paddingBottom: 80 },
  title: { fontSize: 22, fontWeight: '700', color: theme.text, marginBottom: 16 },
  content: { fontSize: 16, lineHeight: 26, color: theme.text, minHeight: 240 },
  preview: { flex: 1, backgroundColor: theme.card },
  previewContent: { padding: 20, paddingBottom: 80 },
  previewTitle: { fontSize: 24, fontWeight: '700', color: theme.text, marginBottom: 16 },
  statusBar: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    paddingHorizontal: 16,
    paddingVertical: 8,
    backgroundColor: theme.bgSoft,
    borderTopWidth: 1,
    borderTopColor: theme.border,
  },
  statusText: { fontSize: 11, color: theme.textLight },
  modalOverlay: { flex: 1, backgroundColor: 'rgba(0,0,0,0.5)', justifyContent: 'flex-end', zIndex: 9998,
elevation: 9998 },
  actionSheet: {
    backgroundColor: theme.card,
    borderTopLeftRadius: 20,
    borderTopRightRadius: 20,
    padding: 12,
    paddingBottom: 32,
    zIndex: 9999,
    elevation: 9999,
  },
  actionItem: {
    flexDirection: 'row',
    alignItems: 'center',
    paddingVertical: 14,
    paddingHorizontal: 12,

    gap: 14,
    borderBottomWidth: 1,
    borderBottomColor: theme.border,
  },
  actionDanger: { borderBottomWidth: 0 },
  actionEmoji: { fontSize: 18 },
  actionText: { flex: 1, fontSize: 15, color: theme.text },
  tagSheet: {
    backgroundColor: theme.card,
    borderTopLeftRadius: 20,
    borderTopRightRadius: 20,
    padding: 20,
    paddingBottom: 32,
    maxHeight: '70%',
    zIndex: 9999,
```

```
    elevation: 9999,
  },
  sheetHeader: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'baseline', marginBottom: 14
},
  sheetTitle: { fontSize: 16, fontWeight: '700', color: theme.text },
  sheetHint: { fontSize: 12, color: theme.textLight },
  tagsList: { flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginBottom: 16 },
  emptyTags: { color: theme.textLight, fontSize: 13, paddingVertical: 20, textAlign: 'center', width: '100%' },
  addTagRow: { flexDirection: 'row', gap: 10 },
  addTagInput: {
    flex: 1,
    borderWidth: 1,
    borderColor: theme.border,
    borderRadius: 10,
    paddingHorizontal: 12,
    height: 42,
    fontSize: 14,
    color: theme.text,
    backgroundColor: theme.bg,
  },
  addTagBtn: {
    backgroundColor: theme.primary,
    borderRadius: 10,
    paddingHorizontal: 16,
    alignItems: 'center',
    justifyContent: 'center',
  },
  addTagBtnText: { color: '#fff', fontWeight: '700' },
})

// ==================== mobile/src/screens/TrashScreen.tsx ====================
import React, { useCallback } from 'react'
import { View, Text, FlatList, TouchableOpacity, StyleSheet, RefreshControl } from 'react-native'
import { useFocusEffect } from '@react-navigation/native'
import { useAuthStore } from '../stores/authStore'
import { useNotesStore } from '../stores/noteStore'

import NoteCard from '../components/NoteCard'
import EmptyState from '../components/EmptyState'
import { toast } from '../stores/toastStore'
import { confirm } from '../stores/confirmStore'
import { theme, timeAgo } from '../theme'

export default function TrashScreen() {
  const token = useAuthStore((s) => s.token)!
  const { items, loading, fetchList, restoreNote, removeNote, emptyTrash } = useNotesStore()

  useFocusEffect(
    useCallback(() => {
      fetchList(token, { trashed: true })
    }, [token]),
```

```
  )

  async function onEmpty() {
    if (items.length === 0) return
    const ok = await confirm({
      title: '清空回收站',
      message: '彻底删除所有回收站笔记？此操作不可撤销',
      confirmText: '清空',
      danger: true,
    })
    if (!ok) return
    try {
      const count = await emptyTrash(token)
      toast.success(`已清空，删除了 ${count} 篇笔记`)
    } catch (e: any) {
      toast.error('操作失败：' + e.message)
    }
  }

  return (
    <View style={styles.wrap}>
      <View style={styles.header}>
        <View style={styles.headerLeft}>
          <Text style={styles.headerEmoji}>🗑</Text>
          <View>
            <Text style={styles.headerTitle}>回收站</Text>
            <Text style={styles.headerHint}>{items.length} 篇已删除笔记</Text>
          </View>
        </View>
        {items.length > 0 && (
          <TouchableOpacity onPress={onEmpty} style={styles.emptyBtn}>
            <Text style={styles.emptyBtnText}>清空</Text>
          </TouchableOpacity>
        )}
      </View>


      <FlatList
        style={styles.list}
        contentContainerStyle={{ flexGrow: 1, padding: 16, paddingTop: 8 }}
        data={items}
        keyExtractor={(item) => item.id}
        refreshControl={
          <RefreshControl refreshing={loading} onRefresh={() => fetchList(token, { trashed: true })} tintColor=
{theme.primary} colors={[theme.primary]} />
        }
        ListEmptyComponent={
          !loading ? <EmptyState emoji="♻️" text="回收站为空" hint="删除的笔记会在这里保留，可随时恢复" /> :
null
        }
        renderItem={({ item }) => (
          <View style={styles.cardWrap}>
```

```
        <View style={styles.deletedBadge}>
          <Text style={styles.deletedBadgeText}>已删除 • {timeAgo(item.deletedAt!)}</Text>
        </View>
        <NoteCard note={item} />
        <View style={styles.actions}>
          <TouchableOpacity
            style={[styles.actionBtn, styles.restoreBtn]}
            onPress={async () => {
              const ok = await confirm({
                title: '恢复笔记',
                message: `恢复「${item.title || '无标题'}」到列表？`,
                confirmText: '恢复',
              })
              if (!ok) return
              try {
                await restoreNote(token, item.id)
                toast.success('已恢复')
              } catch (e: any) {
                toast.error('恢复失败：' + e.message)
              }
            }}
          >
            <Text style={styles.restoreText}>↩ 恢复</Text>
          </TouchableOpacity>
          <TouchableOpacity
            style={[styles.actionBtn, styles.permanentBtn]}
            onPress={async () => {
              const ok = await confirm({
                title: '永久删除',
                message: `永久删除「${item.title || '无标题'}」？此操作不可撤销`,
                confirmText: '永久删除',
                danger: true,
              })
              if (!ok) return
              try {
                await removeNote(token, item.id)
                toast.success('已永久删除')

              } catch (e: any) {
                toast.error('删除失败：' + e.message)
              }
            }}
          >
            <Text style={styles.permanentText}>永久删除</Text>
          </TouchableOpacity>
        </View>
      </View>
    )}
  />
</View>
```

```
  )
}

const styles = StyleSheet.create({
  wrap: { flex: 1, backgroundColor: theme.bg },
  header: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'center',
    padding: 16,
    backgroundColor: theme.card,
    borderBottomWidth: 1,
    borderBottomColor: theme.border,
  },
  headerLeft: { flexDirection: 'row', alignItems: 'center', gap: 12 },
  headerEmoji: { fontSize: 28 },
  headerTitle: { fontSize: 16, fontWeight: '700', color: theme.text },
  headerHint: { fontSize: 12, color: theme.textMuted, marginTop: 2 },
  emptyBtn: {
    paddingHorizontal: 14,
    paddingVertical: 8,
    borderRadius: 999,
    backgroundColor: theme.danger + '18',
  },
  emptyBtnText: { color: theme.danger, fontSize: 13, fontWeight: '700' },
  list: { flex: 1 },
  cardWrap: { marginBottom: 12, position: 'relative' },
  deletedBadge: {
    position: 'absolute',
    top: 8,
    right: 12,
    zIndex: 1,
    paddingHorizontal: 8,
    paddingVertical: 3,
    borderRadius: 6,
    backgroundColor: theme.danger + '22',
  },
  deletedBadgeText: { color: theme.danger, fontSize: 10, fontWeight: '600' },

  actions: {
    flexDirection: 'row',
    gap: 10,
    paddingHorizontal: 4,
    marginTop: 8,
  },
  actionBtn: { paddingHorizontal: 14, paddingVertical: 8, borderRadius: 8, borderWidth: 1.2 },
  restoreBtn: { borderColor: theme.primary, backgroundColor: theme.primarySoft },
  restoreText: { color: theme.primary, fontSize: 13, fontWeight: '600' },
  permanentBtn: { borderColor: theme.danger + '55', backgroundColor: theme.danger + '10' },
  permanentText: { color: theme.danger, fontSize: 13, fontWeight: '600' },
```

```
})

// ==================== mobile/src/screens/SettingsScreen.tsx ====================
import React, { useState, useEffect } from 'react'
import { View, Text, TextInput, TouchableOpacity, StyleSheet, ScrollView, Modal, Pressable } from 'react-native'
import { useAuthStore } from '../stores/authStore'
import { useNotesStore } from '../stores/noteStore'
import { api } from '../lib/api'
import TagChip from '../components/TagChip'
import { toast } from '../stores/toastStore'
import { confirm } from '../stores/confirmStore'
import { theme, initials, colorFromString } from '../theme'

export default function SettingsScreen() {
  const { user, logout, updateProfile } = useAuthStore()
  const token = useAuthStore((s) => s.token)!
  const { tags, fetchTags, createTag, updateNote } = useNotesStore() // updateNote 占位避免警告
  void updateNote
  const [nickname, setNickname] = useState(user?.nickname || '')
  const [newTag, setNewTag] = useState('')
  const [tagColor, setTagColor] = useState(theme.tagColors[0])
  const [oldPwd, setOldPwd] = useState('')
  const [newPwd, setNewPwd] = useState('')
  const [confirmPwd, setConfirmPwd] = useState('')
  const [editingTag, setEditingTag] = useState<{ id: string; name: string; color: string } | null>(null)

  useEffect(() => {
    fetchTags(token)
  }, [token])

  async function saveProfile() {
    if (!nickname.trim()) return toast.error('昵称不能为空')
    try {
      await updateProfile({ nickname: nickname.trim() })
      toast.success('资料已更新')
    } catch (e: any) {
      toast.error('更新失败：' + e.message)
    }
  }


  async function addTag() {
    const name = newTag.trim()
    if (!name) return
    try {
      await createTag(token, name)
      setNewTag('')
      toast.success('已添加标签：' + name)
    } catch (e: any) {
      toast.error('添加失败：' + e.message)
```

```
  }
}

async function saveTagEdit() {
  if (!editingTag) return
  try {
    await api.tags.update(token, editingTag.id, { name: editingTag.name, color: editingTag.color })
    await fetchTags(token)
    setEditingTag(null)
    toast.success('标签已更新')
  } catch (e: any) {
    toast.error('保存失败：' + e.message)
  }
}

async function deleteTag(id: string, name: string) {
  const ok = await confirm({
    title: '删除标签',
    message: `确定删除「${name}」？该标签会从所有笔记中移除`,
    confirmText: '删除',
    danger: true,
  })
  if (!ok) return
  try {
    await api.tags.remove(token, id)
    await fetchTags(token)
    toast.success('已删除标签')
  } catch (e: any) {
    toast.error('删除失败：' + e.message)
  }
}

async function changePassword() {
  if (!oldPwd || !newPwd) return toast.error('请填写原密码和新密码')
  if (newPwd.length < 6) return toast.error('新密码至少 6 位')
  if (newPwd !== confirmPwd) return toast.error('两次密码不一致')
  try {
    const res = await api.auth.updatePassword(token, { oldPassword: oldPwd, newPassword: newPwd })
    toast.success(res.message)
    setOldPwd('')

    setNewPwd('')
    setConfirmPwd('')
  } catch (e: any) {
    toast.error('修改失败：' + e.message)
  }
}

async function onLogout() {
  const ok = await confirm({
```

```
      title: '退出登录',
      message: '确定要退出当前账号吗？',
      confirmText: '退出',
      danger: true,
    })
    if (ok) logout()
  }

  const userInitial = initials(user?.nickname, user?.email)
  const avatarColor = colorFromString(user?.username || user?.email || 'X')

  return (
    <ScrollView style={styles.wrap} contentContainerStyle={{ padding: 16, paddingBottom: 60 }}>
      {/* 用户卡片 */}
      <View style={styles.profileCard}>
        <View style={[styles.avatar, { backgroundColor: avatarColor }]}>
          <Text style={styles.avatarText}>{userInitial}</Text>
        </View>
        <View style={styles.profileInfo}>
          <Text style={styles.nickname}>{user?.nickname || user?.username}</Text>
          <Text style={styles.email}>{user?.email}</Text>
        </View>
      </View>

      {/* 资料修改 */}
      <View style={styles.section}>
        <View style={styles.sectionHeader}>
          <Text style={styles.sectionTitle}>个人资料</Text>
        </View>
        <Text style={styles.label}>昵称</Text>
        <TextInput style={styles.input} value={nickname} onChangeText={setNickname} placeholder="设置昵称"
placeholderTextColor={theme.textLight} />
        <TouchableOpacity style={styles.btn} onPress={saveProfile}>
          <Text style={styles.btnText}>保存</Text>
        </TouchableOpacity>
      </View>

      {/* 标签管理 */}
      <View style={styles.section}>
        <View style={styles.sectionHeader}>
          <Text style={styles.sectionTitle}>标签管理</Text>
          <Text style={styles.sectionHint}>{tags.length} 个</Text>

        </View>
        <View style={styles.tagsGrid}>
          {tags.map((t) => (
            <TouchableOpacity
              key={t.id}
              onLongPress={() => setEditingTag({ id: t.id, name: t.name, color: t.color || theme.primary })}
              onPress={() => setEditingTag({ id: t.id, name: t.name, color: t.color || theme.primary })}
            >
```

```
              <TagChip name={t.name} color={t.color} />
            </TouchableOpacity>
          ))}
          {tags.length === 0 && <Text style={styles.empty}>还没有标签，添加一个吧</Text>}
        </View>
        <View style={styles.tagCreator}>
          <TextInput
            style={[styles.input, { flex: 1 }]}
            placeholder="新标签名"
            placeholderTextColor={theme.textLight}
            value={newTag}
            onChangeText={setNewTag}
          />
          <TouchableOpacity style={[styles.colorDot, { backgroundColor: tagColor }]} onPress={() => {
            const idx = theme.tagColors.indexOf(tagColor)
            setTagColor(theme.tagColors[(idx + 1) % theme.tagColors.length])
          }} />
          <TouchableOpacity style={[styles.btn, { marginLeft: 0, marginTop: 0, paddingHorizontal: 16,
paddingVertical: 12, alignSelf: 'center' }]} onPress={addTag}>
            <Text style={styles.btnText}>添加</Text>
          </TouchableOpacity>
        </View>
        <Text style={styles.tip}>提示：点击颜色块切换标签颜色 • 点击标签可重命名/删除</Text>
      </View>

      {/* 修改密码 */}
      <View style={styles.section}>
        <View style={styles.sectionHeader}>
          <Text style={styles.sectionTitle}>修改密码</Text>
        </View>
        <TextInput style={[styles.input, { marginBottom: 10 }]} placeholder="原密码" secureTextEntry value=
{oldPwd} onChangeText={setOldPwd} placeholderTextColor={theme.textLight} />
        <TextInput style={[styles.input, { marginBottom: 10 }]} placeholder="新密码（至少 6 位）" secureTextEntry
value={newPwd} onChangeText={setNewPwd} placeholderTextColor={theme.textLight} />
        <TextInput style={[styles.input, { marginBottom: 10 }]} placeholder="确认新密码" secureTextEntry value=
{confirmPwd} onChangeText={setConfirmPwd} placeholderTextColor={theme.textLight} />
        <TouchableOpacity style={styles.btn} onPress={changePassword}>
          <Text style={styles.btnText}>修改密码</Text>
        </TouchableOpacity>
      </View>

      {/* 关于 */}
      <View style={styles.section}>
        <View style={styles.aboutRow}>
          <Text style={styles.aboutLabel}>版本</Text>
          <Text style={styles.aboutValue}>1.0.0</Text>

        </View>
        <View style={[styles.aboutRow, { alignItems: 'flex-start' }]}>
          <Text style={styles.aboutLabel}>简介</Text>
          <Text style={[styles.aboutValue, { flexShrink: 1, flexWrap: 'wrap', maxWidth: '70%' }]}>富商笔记，记录
每一刻灵感，随时随地同步你的笔记。</Text>
        </View>
      </View>
```

```
      <TouchableOpacity style={styles.logoutBtn} onPress={onLogout}>
        <Text style={styles.logoutText}>退出登录</Text>
      </TouchableOpacity>

      {/* 编辑标签 Modal */}
      <Modal visible={!!editingTag} transparent animationType="fade" onRequestClose={() => setEditingTag(null)}>
        <Pressable style={styles.modalOverlay} onPress={() => setEditingTag(null)}>
          <Pressable style={styles.editSheet} onPress={(e) => e.stopPropagation()}>
            <Text style={styles.sheetTitle}>编辑标签</Text>
            <Text style={styles.label}>名称</Text>
            <TextInput
              style={styles.input}
              value={editingTag?.name}
              onChangeText={(v) => setEditingTag((p) => (p ? { ...p, name: v } : p))}
              placeholder="标签名"
              placeholderTextColor={theme.textLight}
            />
            <Text style={styles.label}>颜色</Text>
            <View style={styles.colorPicker}>
              {theme.tagColors.map((c) => (
                <TouchableOpacity
                  key={c}
                  style={[styles.colorDot, { backgroundColor: c, borderColor: editingTag?.color === c ? '#0f172a'
: 'transparent' }]}
                  onPress={() => setEditingTag((p) => (p ? { ...p, color: c } : p))}
                />
              ))}
            </View>
            <View style={styles.sheetActions}>
              <TouchableOpacity
                style={[styles.sheetBtn, { backgroundColor: theme.danger }]}
                onPress={async () => {
                  if (!editingTag) return
                  const tagData = editingTag
                  setEditingTag(null)
                  await deleteTag(tagData.id, tagData.name)
                }}
              >
                <Text style={styles.btnText}>删除</Text>
              </TouchableOpacity>
              <TouchableOpacity style={[styles.sheetBtn, { flex: 1 }]} onPress={saveTagEdit}>
                <Text style={styles.btnText}>保存</Text>
              </TouchableOpacity>
            </View>

          </Pressable>
        </Pressable>
      </Modal>
    </ScrollView>
  )
}
```

```
const styles = StyleSheet.create({
  wrap: { flex: 1, backgroundColor: theme.bg },
  profileCard: {
    backgroundColor: theme.primary,
    borderRadius: 20,
    padding: 20,
    flexDirection: 'row',
    alignItems: 'center',
    gap: 16,
    marginBottom: 16,
    ...theme.shadow.md,
  },
  avatar: {
    width: 56,
    height: 56,
    borderRadius: 28,
    alignItems: 'center',
    justifyContent: 'center',
  },
  avatarText: { color: '#fff', fontSize: 24, fontWeight: '700' },
  profileInfo: { flex: 1 },
  nickname: { color: '#fff', fontSize: 18, fontWeight: '700' },
  email: { color: '#e0e7ff', fontSize: 13, marginTop: 4 },
  section: {
    backgroundColor: theme.card,
    borderRadius: 16,
    padding: 16,
    marginBottom: 12,
    borderWidth: 1,
    borderColor: theme.border,
  },
  sectionHeader: {
    flexDirection: 'row',
    justifyContent: 'space-between',
    alignItems: 'baseline',
    marginBottom: 14,
  },
  sectionTitle: { fontSize: 15, fontWeight: '700', color: theme.text },
  sectionHint: { fontSize: 12, color: theme.textLight },
  label: { fontSize: 12, color: theme.textMuted, marginBottom: 6, marginTop: 8, fontWeight: '500' },
  input: {
    backgroundColor: theme.bg,
    borderWidth: 1.5,

    borderColor: theme.border,
    borderRadius: 10,
    paddingHorizontal: 14,
    height: 44,
    fontSize: 14,
```

```
    color: theme.text,
  },
  btn: {
    backgroundColor: theme.primary,
    borderRadius: 10,
    paddingVertical: 12,
    alignItems: 'center',
    marginTop: 12,
    ...theme.shadow.sm,
  },
  btnText: { color: '#fff', fontWeight: '700', fontSize: 14 },
  tagsGrid: { flexDirection: 'row', flexWrap: 'wrap', gap: 8, marginBottom: 12 },
  empty: { color: theme.textLight, fontSize: 13, paddingVertical: 12 },
  tagCreator: { flexDirection: 'row', alignItems: 'center', gap: 8 },
  colorDot: {
    width: 28,
    height: 28,
    borderRadius: 14,
    borderWidth: 2,
  },
  tip: { fontSize: 11, color: theme.textLight, marginTop: 10, lineHeight: 16 },
  aboutRow: { flexDirection: 'row', justifyContent: 'space-between', paddingVertical: 10, borderBottomWidth: 1,
borderBottomColor: theme.border },
  aboutLabel: { color: theme.textMuted, fontSize: 14 },
  aboutValue: { color: theme.text, fontSize: 14, fontWeight: '500' },
  logoutBtn: {
    backgroundColor: theme.card,
    borderRadius: 14,
    paddingVertical: 14,
    alignItems: 'center',
    borderWidth: 1,
    borderColor: theme.danger + '44',
    marginTop: 4,
  },
  logoutText: { color: theme.danger, fontWeight: '700', fontSize: 15 },
  modalOverlay: { flex: 1, backgroundColor: 'rgba(0,0,0,0.5)', justifyContent: 'center', padding: 24, zIndex:
9998, elevation: 9998 },
  editSheet: {
    backgroundColor: theme.card,
    borderRadius: 16,
    padding: 20,
    zIndex: 9999,
    elevation: 9999,
  },
  sheetTitle: { fontSize: 16, fontWeight: '700', color: theme.text, marginBottom: 12, textAlign: 'center' },
  colorPicker: { flexDirection: 'row', gap: 10, justifyContent: 'center', paddingVertical: 10 },
  sheetActions: { flexDirection: 'row', gap: 10, marginTop: 16 },

  sheetBtn: {
    flex: 1,
    borderRadius: 10,
    paddingVertical: 12,
```

```
    alignItems: 'center',
    backgroundColor: theme.primary,
  },
})

// ==================== server/prisma/schema.prisma ====================
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "mysql"
  url      = env("DATABASE_URL")
}

model User {
  id           String   @id @default(uuid())
  email        String   @unique
  username     String   @unique
  passwordHash String
  nickname     String?
  avatar       String?
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt
  notes        Note[]
  tags         Tag[]
}

model Tag {
  id        String   @id @default(uuid())
  name      String
  color     String   @default("#3b82f6")
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  notes     Note[]
  createdAt DateTime @default(now())

  @@unique([userId, name])
  @@index([userId])
}

model Note {
  id          String    @id @default(uuid())
  title       String    @default("")
  content     String    @db.LongText
  userId      String

  user        User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  tags        Tag[]
  pinned      Boolean   @default(false)
```

```
  deletedAt   DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  @@index([userId, deletedAt])
  @@index([userId, updatedAt])
  @@index([userId, pinned])
}

// 扩展：简单的刷新令牌黑名单（注销下线）
model RevokedToken {
  id        String   @id @default(uuid())
  token     String   @unique
  userId    String
  createdAt DateTime @default(now())

  @@index([userId])
}

// ==================== server/prisma/seed.ts ====================
import { PrismaClient } from '@prisma/client'
import bcrypt from 'bcryptjs'

const prisma = new PrismaClient()

async function main() {
  const passwordHash = await bcrypt.hash('123456', 10)
  const user = await prisma.user.upsert({
    where: { email: 'demo@clound.note' },
    update: {},
    create: {
      email: 'demo@clound.note',
      username: 'demo',
      passwordHash,
      nickname: '演示账号',
    },
  })

  const tags = ['工作', '生活', '灵感']
  for (const name of tags) {
    await prisma.tag.upsert({
      where: { userId_name: { userId: user.id, name } },
      update: {},
      create: { name, userId: user.id },
    })
  }


  const workTag = await prisma.tag.findFirst({ where: { userId: user.id, name: '工作' } })
  await prisma.note.create({
```

```
    data: {
      title: '欢迎使用富商笔记',
      content: '# 欢迎\n\n这是你的第一篇笔记，可以开始编辑了。',
      userId: user.id,
      pinned: true,
      tags: workTag ? { connect: [{ id: workTag.id }] } : undefined,
    },
  })

  console.log('seed done, user:', user.email)
}

main()
  .catch((e) => {
    console.error(e)
    process.exit(1)
  })
  .finally(() => prisma.$disconnect())

// ==================== server/src/utils/ApiError.ts ====================
export class ApiError extends Error {
  constructor(
    public status: number,
    public code: string,
    message: string,
  ) {
    super(message)
    this.name = 'ApiError'
  }

  static badRequest(code: string, message: string) {
    return new ApiError(400, code, message)
  }

  static unauthorized(message = '未登录或登录已过期') {
    return new ApiError(401, 'UNAUTHORIZED', message)
  }

  static forbidden(message = '无权访问') {
    return new ApiError(403, 'FORBIDDEN', message)
  }

  static notFound(message = '资源不存在') {
    return new ApiError(404, 'NOT_FOUND', message)
  }

  static conflict(code: string, message: string) {
    return new ApiError(409, code, message)

  }
```

```
}

// ==================== server/src/utils/jwt.ts ====================
import jwt, { type SignOptions } from 'jsonwebtoken'
import { z } from 'zod'

const secret = process.env.JWT_SECRET || 'dev-secret-change-me'
const expiresIn = process.env.JWT_EXPIRES_IN || '7d'

export const TokenPayload = z.object({
  userId: z.string(),
  email: z.string(),
})

export type TokenPayloadType = z.infer<typeof TokenPayload>

export function signToken(payload: TokenPayloadType): string {
  const options = { expiresIn } as SignOptions
  return jwt.sign(payload, secret, options)
}

export function verifyToken(token: string): TokenPayloadType | null {
  try {
    const decoded = jwt.verify(token, secret) as Record<string, unknown>
    const parsed = TokenPayload.safeParse(decoded)
    return parsed.success ? parsed.data : null
  } catch {
    return null
  }
}

// ==================== server/src/lib/prisma.ts ====================
import { PrismaClient } from '@prisma/client'

export const prisma = new PrismaClient({
  log: process.env.NODE_ENV === 'development' ? ['query', 'error', 'warn'] : ['error'],
})

// ==================== server/src/middlewares/auth.ts ====================
import type { Request, Response, NextFunction } from 'express'
import { verifyToken } from '../utils/jwt'
import { ApiError } from '../utils/ApiError'

export interface AuthRequest extends Request {
  userId?: string
  userEmail?: string
}

export async function authMiddleware(req: AuthRequest, _res: Response, next: NextFunction) {
```

```
  const header = req.headers.authorization
  if (!header || !header.startsWith('Bearer ')) {
    return next(ApiError.unauthorized())
  }

  const token = header.slice(7).trim()
  const payload = verifyToken(token)
  if (!payload) {
    return next(ApiError.unauthorized())
  }

  req.userId = payload.userId
  req.userEmail = payload.email
  next()
}

// ==================== server/src/middlewares/error.ts ====================
import type { Request, Response, NextFunction } from 'express'
import { ZodError } from 'zod'
import { ApiError } from '../utils/ApiError'

type ErrorLike = Error & { code?: string; status?: number }

export function errorHandler(err: ErrorLike, _req: Request, res: Response, _next: NextFunction) {
  if (err instanceof ApiError) {
    return res.status(err.status).json({ code: err.code, message: err.message })
  }

  if (err instanceof ZodError) {
    const first = err.errors[0]
    return res.status(400).json({
      code: 'VALIDATION_ERROR',
      message: first ? `${first.path.join('.')}: ${first.message}` : '参数校验失败',
    })
  }

  if (err.code === 'P2002') {
    return res.status(409).json({ code: 'CONFLICT', message: '字段已存在' })
  }

  console.error('[server error]', err)
  res.status(err.status || 500).json({
    code: 'SERVER_ERROR',
    message: err.message || '服务器内部错误',
  })
}

// ==================== server/src/routes/auth.ts ====================
import { Router, type Response } from 'express'
import bcrypt from 'bcryptjs'
```

```
import { z } from 'zod'
import { prisma } from '../lib/prisma'
import { signToken } from '../utils/jwt'
import { ApiError } from '../utils/ApiError'
import { authMiddleware, type AuthRequest } from '../middlewares/auth'

const router = Router()

const registerSchema = z.object({
  email: z.string().email('邮箱格式不正确'),
  username: z.string().min(2, '用户名至少 2 个字符').max(24, '用户名最多 24 个字符'),
  password: z.string().min(6, '密码至少 6 位').max(64, '密码过长'),
  nickname: z.string().max(32).optional(),
})

const loginSchema = z.object({
  account: z.string().min(2, '请输入邮箱或用户名'),
  password: z.string().min(1, '请输入密码'),
})

router.post('/register', async (req: AuthRequest, res: Response, next) => {
  try {
    const body = registerSchema.parse(req.body)
    const passwordHash = await bcrypt.hash(body.password, 10)

    const exists = await prisma.user.findFirst({
      where: { OR: [{ email: body.email }, { username: body.username }] },
      select: { email: true, username: true },
    })
    if (exists) {
      throw ApiError.conflict(
        'EMAIL_EXISTS',
        exists.email === body.email ? '邮箱已被注册' : '用户名已被占用',
      )
    }

    const user = await prisma.user.create({
      data: {
        email: body.email,
        username: body.username,
        passwordHash,
        nickname: body.nickname || body.username,
      },
      select: { id: true, email: true, username: true, nickname: true, avatar: true },
    })

    const token = signToken({ userId: user.id, email: user.email })
    res.status(201).json({ token, user })
  } catch (e) {
```

```
    next(e)

  }
})

router.post('/login', async (req: AuthRequest, res: Response, next) => {
  try {
    const body = loginSchema.parse(req.body)
    const user = await prisma.user.findFirst({
      where: { OR: [{ email: body.account }, { username: body.account }] },
    })
    if (!user) throw ApiError.unauthorized('账号或密码错误')

    const ok = await bcrypt.compare(body.password, user.passwordHash)
    if (!ok) throw ApiError.unauthorized('账号或密码错误')

    const token = signToken({ userId: user.id, email: user.email })
    res.json({
      token,
      user: {
        id: user.id,
        email: user.email,
        username: user.username,
        nickname: user.nickname,
        avatar: user.avatar,
      },
    })
  } catch (e) {
    next(e)
  }
})

router.get('/me', authMiddleware, async (req: AuthRequest, res: Response, next) => {
  try {
    const user = await prisma.user.findUnique({
      where: { id: req.userId! },
      select: { id: true, email: true, username: true, nickname: true, avatar: true, createdAt: true },
    })
    if (!user) throw ApiError.notFound('用户不存在')
    res.json({ user })
  } catch (e) {
    next(e)
  }
})

router.put('/profile', authMiddleware, async (req: AuthRequest, res: Response, next) => {
  try {
    const body = z
      .object({
        nickname: z.string().min(1).max(32).optional(),
```

```
        avatar: z.string().url().optional(),
      })

      .parse(req.body)

    const user = await prisma.user.update({
      where: { id: req.userId! },
      data: { nickname: body.nickname, avatar: body.avatar },
      select: { id: true, email: true, username: true, nickname: true, avatar: true },
    })
    res.json({ user })
  } catch (e) {
    next(e)
  }
})

router.put('/password', authMiddleware, async (req: AuthRequest, res: Response, next) => {
  try {
    const body = z
      .object({
        oldPassword: z.string().min(1),
        newPassword: z.string().min(6).max(64),
      })
      .parse(req.body)

    const user = await prisma.user.findUnique({ where: { id: req.userId! } })
    if (!user) throw ApiError.notFound('用户不存在')

    const ok = await bcrypt.compare(body.oldPassword, user.passwordHash)
    if (!ok) throw ApiError.badRequest('WRONG_PASSWORD', '原密码错误')

    const passwordHash = await bcrypt.hash(body.newPassword, 10)
    await prisma.user.update({ where: { id: user.id }, data: { passwordHash } })
    res.json({ message: '密码已更新' })
  } catch (e) {
    next(e)
  }
})

export default router

// ==================== server/src/routes/notes.ts ====================
import { Router, type Response } from 'express'
import { z } from 'zod'
import { prisma } from '../lib/prisma'
import { ApiError } from '../utils/ApiError'
import { authMiddleware, type AuthRequest } from '../middlewares/auth'

const router = Router()
```

```
router.use(authMiddleware)

const listQuerySchema = z.object({

  keyword: z.string().optional(),
  tagId: z.string().optional(),
  trashed: z.enum(['true', 'false']).default('false'),
  pinnedOnly: z.enum(['true', 'false']).default('false'),
  limit: z.coerce.number().int().min(1).max(100).default(50),
  offset: z.coerce.number().int().min(0).default(0),
})

router.get('/', async (req: AuthRequest, res: Response, next) => {
  try {
    const q = listQuerySchema.parse(req.query)
    const where: any = { userId: req.userId! }
    if (q.trashed === 'true') {
      where.deletedAt = { not: null }
    } else {
      where.deletedAt = null
      if (q.pinnedOnly === 'true') where.pinned = true
    }
    if (q.keyword) {
      const kw = q.keyword.trim()
      if (kw) {
        where.OR = [
          { title: { contains: kw } },
          { content: { contains: kw } },
        ]
      }
    }
    if (q.tagId) where.tags = { some: { id: q.tagId } }

    const [items, total] = await Promise.all([
      prisma.note.findMany({
        where,
        orderBy: [{ pinned: 'desc' }, { updatedAt: 'desc' }],
        take: q.limit,
        skip: q.offset,
        include: { tags: true },
      }),
      prisma.note.count({ where }),
    ])

    res.json({ items, total })
  } catch (e) {
    next(e)
  }
})
```

```
router.get('/:id', async (req: AuthRequest, res: Response, next) => {
  try {
    const note = await prisma.note.findFirst({
      where: { id: req.params.id, userId: req.userId! },

      include: { tags: true },
    })
    if (!note) throw ApiError.notFound('笔记不存在')
    res.json({ note })
  } catch (e) {
    next(e)
  }
})

const createSchema = z.object({
  title: z.string().max(200).default(''),
  content: z.string().default(''),
  pinned: z.boolean().default(false),
  tagIds: z.array(z.string()).default([]),
})

router.post('/', async (req: AuthRequest, res: Response, next) => {
  try {
    const body = createSchema.parse(req.body)
    const note = await prisma.note.create({
      data: {
        title: body.title,
        content: body.content ?? '',
        pinned: body.pinned,
        userId: req.userId!,
        tags: body.tagIds.length
          ? { connect: body.tagIds.map((id) => ({ id })) }
          : undefined,
      },
      include: { tags: true },
    })
    res.status(201).json({ note })
  } catch (e) {
    next(e)
  }
})

const updateSchema = z.object({
  title: z.string().max(200).optional(),
  content: z.string().optional(),
  pinned: z.boolean().optional(),
  tagIds: z.array(z.string()).optional(),
})

router.put('/:id', async (req: AuthRequest, res: Response, next) => {
```

```
  try {
    const body = updateSchema.parse(req.body)
    const existing = await prisma.note.findFirst({
      where: { id: req.params.id, userId: req.userId! },
      select: { id: true, deletedAt: true },

    })
    if (!existing) throw ApiError.notFound('笔记不存在')

    const data: any = {}
    if (body.title !== undefined) data.title = body.title
    if (body.content !== undefined) data.content = body.content
    if (body.pinned !== undefined) data.pinned = body.pinned
    if (body.tagIds !== undefined) {
      data.tags = { set: body.tagIds.map((id) => ({ id })) }
    }

    const note = await prisma.note.update({
      where: { id: existing.id },
      data,
      include: { tags: true },
    })
    res.json({ note })
  } catch (e) {
    next(e)
  }
})

router.delete('/:id', async (req: AuthRequest, res: Response, next) => {
  try {
    const note = await prisma.note.findFirst({
      where: { id: req.params.id, userId: req.userId! },
      select: { id: true, deletedAt: true },
    })
    if (!note) throw ApiError.notFound('笔记不存在')

    if (!note.deletedAt) {
      await prisma.note.update({
        where: { id: note.id },
        data: { deletedAt: new Date(), pinned: false },
      })
      return res.json({ message: '已移入回收站' })
    }
    await prisma.note.delete({ where: { id: note.id } })
    res.json({ message: '已彻底删除' })
  } catch (e) {
    next(e)
  }
})
```

```
router.post('/:id/restore', async (req: AuthRequest, res: Response, next) => {
  try {
    const note = await prisma.note.findFirst({
      where: { id: req.params.id, userId: req.userId!, deletedAt: { not: null } },
      select: { id: true },
    })

    if (!note) throw ApiError.notFound('笔记不在回收站')
    const restored = await prisma.note.update({
      where: { id: note.id },
      data: { deletedAt: null },
      include: { tags: true },
    })
    res.json({ note: restored, message: '已恢复' })
  } catch (e) {
    next(e)
  }
})

router.delete('/trash/empty', async (req: AuthRequest, res: Response, next) => {
  try {
    const result = await prisma.note.deleteMany({
      where: { userId: req.userId!, deletedAt: { not: null } },
    })
    res.json({ message: '回收站已清空', count: result.count })
  } catch (e) {
    next(e)
  }
})

export default router

// ==================== server/src/routes/tags.ts ====================
import { Router, type Response } from 'express'
import { z } from 'zod'
import { prisma } from '../lib/prisma'
import { ApiError } from '../utils/ApiError'
import { authMiddleware, type AuthRequest } from '../middlewares/auth'

const router = Router()

router.use(authMiddleware)

router.get('/', async (req: AuthRequest, res: Response, next) => {
  try {
    const tags = await prisma.tag.findMany({
      where: { userId: req.userId! },
      orderBy: { createdAt: 'asc' },
      include: { _count: { select: { notes: true } } },
    })
```

```
    res.json({ tags })
  } catch (e) {
    next(e)
  }
})

const createSchema = z.object({

  name: z.string().min(1, '标签名不能为空').max(32, '标签名过长'),
  color: z.string().regex(/^#[0-9a-fA-F]{6}$/, '颜色格式不正确').optional(),
})

router.post('/', async (req: AuthRequest, res: Response, next) => {
  try {
    const body = createSchema.parse(req.body)
    const exists = await prisma.tag.findUnique({
      where: { userId_name: { userId: req.userId!, name: body.name } },
      select: { id: true },
    })
    if (exists) throw ApiError.conflict('TAG_EXISTS', '标签名已存在')
    const tag = await prisma.tag.create({
      data: { name: body.name, color: body.color, userId: req.userId! },
    })
    res.status(201).json({ tag })
  } catch (e) {
    next(e)
  }
})

const updateSchema = z.object({
  name: z.string().min(1).max(32).optional(),
  color: z.string().regex(/^#[0-9a-fA-F]{6}$/).optional(),
})

router.put('/:id', async (req: AuthRequest, res: Response, next) => {
  try {
    const body = updateSchema.parse(req.body)
    const tag = await prisma.tag.update({
      where: { id: req.params.id, userId: req.userId! },
      data: { name: body.name, color: body.color },
    })
    res.json({ tag })
  } catch (e) {
    next(e)
  }
})

router.delete('/:id', async (req: AuthRequest, res: Response, next) => {
  try {
    await prisma.tag.delete({ where: { id: req.params.id, userId: req.userId! } })
```